Visitor Management System: Chat Analysis Report

Source: shared ChatGPT conversation "Build Visitor Management App" · Prepared October 25, 2026

1. Executive summary

The conversation documents the user interface design phase of a Visitor Management System built on the Microsoft Power Platform. The app has a Canvas App front end, Dataverse tables for data, a Power Automate flow for notifications, and a Model-Driven App for back-office viewing. The chat covers three pieces of work: a full redesign of the Home screen, a redesign plan for the Visit screen, and a configuration step in the Model-Driven App to show only required columns.

The Home screen is reported as complete. The Visit screen design was proposed, but the chat does not confirm that it was finished. The column view change was explained in steps, with no confirmation either. The chat ends with offers to test the full flow, send email to the Host dynamically, and verify the Power Automate flow.

Four findings stand out:

- The work follows one repeatable pattern on every screen: set the background, add a header, add content, style the existing controls, then test.
- Every instruction protects existing logic. Items formulas, OnSelect formulas, Dataverse tables and the Power Automate flow are explicitly left unchanged.
- One shared palette of six main colours keeps the Home and Visit screens consistent.
- Only one of five tracked work items is confirmed finished, and no end-to-end testing has been done yet.

2. Scope of the analysis

The shared chat contains about eight exchanges. It is almost entirely about UI and presentation. Backend work (Dataverse tables, the Check-In flow, status values) is mentioned only as existing and to be left untouched, so this report cannot describe how those parts were built.

The chat reads as a guided tutorial. The assistant supplies control names, property names and formulas, and the user applies them by hand in Power Apps Studio. The user makes six requests, in this order:

1. Improve the Home screen design.
2. Show a preview of the Home screen with each property explained.
3. Guide the build from the beginning, step by step.
4. Fix a missing Send to back option.
5. Move on to the Visit screen after finishing the Home screen.
6. Show only the required fields in the Model-Driven table.

The chat does not include screenshots, sample data, the Check-In timestamp formula or the definition of the Power Automate flow.

3. Project overview

- **Product:** Visitor Management System, an internal workspace app for registering visitors, managing appointments and tracking check-ins.
- **Front end:** Canvas App with a Home screen and two destination screens (a Visit screen and a Records screen).
- **Data layer:** Dataverse, with a Visit table holding Visit Name, Visitor, Host, Visit Date, Purpose and Status. Check-In and Check-Out times are optional extra columns.
- **Automation:** a Power Automate flow, with email notification to the Host as the next planned improvement.
- **Back office:** a Model-Driven App that lists visits through a table view.
- **Key controls named in the chat:** galVisits (visit gallery) and btnCheckIn (check-in button with a timestamp formula).

The existing controls and data items that the chat protects are listed below.

Item	Role	How the chat treats it
Navigation buttons	Open the Visit and Records screens from Home	Restyled and repositioned; OnSelect formulas kept
galVisits	Gallery that lists visits	Restyled; Items formula kept
btnCheckIn	Marks a visitor as checked in with a timestamp	Restyled; OnSelect formula kept
Visit table	Stores visit records	Columns named: Visit Name, Visitor, Host, Visit Date, Purpose, Status
Power Automate flow	Sends notifications	Left untouched; dynamic email to the Host planned

Item	Role	How the chat treats it
Status values	Describe where a visit stands	Scheduled, Checked In and Completed appear in the target design

4. Design system: Fresh Teal

A single palette is applied across screens so the app looks consistent:

- Primary teal RGBA(15, 118, 110, 1) for headers and primary buttons
- Dark teal RGBA(17, 94, 89, 1) for hover states
- Mint background RGBA(240, 253, 250, 1) for screens
- White RGBA(255, 255, 255, 1) for cards
- Main text RGBA(31, 41, 55, 1) and secondary text RGBA(100, 116, 139, 1)

The chat also sets naming conventions (rectHeader, lblAppName, rectVisitorCard, btnVisitorNavigation and similar) and uses relative sizing such as App.Width and App.Height so layouts adapt to screen size.

Colour	Value	Where it is used
Primary teal	RGBA(15, 118, 110, 1)	Header bars, primary buttons, headings
Dark teal	RGBA(17, 94, 89, 1)	Button hover state
Mint	RGBA(240, 253, 250, 1)	Screen background and label fill
White	RGBA(255, 255, 255, 1)	Cards and button text
Main text	RGBA(31, 41, 55, 1)	Card titles and gallery text
Secondary text	RGBA(100, 116, 139, 1)	Descriptions and footer

Supporting shades also appear: RGBA(204, 251, 241, 1) for card borders and the header subtitle, RGBA(13, 78, 72, 1) for the pressed button state, and RGBA(19, 78, 74, 1) for large headings. Type sizes run from 24 to 30 for titles, 12 to 14 for body text and 10 for the footer.

5. Home screen redesign

The starting point was a plain screen with an app name and two navigation buttons. The redesign turned it into a welcome dashboard in this order: background, teal header, title and subtitle, welcome section, optional illustration, two navigation cards, footer.

- **Header:** a full-width teal rectangle (height 100) with the app title and the tagline "Smart Check-ins • Safe Workplaces".
- **Welcome section:** an eyebrow label, the heading "Manage visits with ease." and a short description.
- **Navigation cards:** two white cards, one for Visitor Management (filled teal button) and one for Visit Records (outlined button).
- **Footer:** a centered label reading "Visitor Management System • Internal Workspace".

A firm rule throughout was to keep the original button OnSelect formulas unchanged, so navigation was never put at risk. The user confirmed the Home screen interface was finished.

The main Home screen controls and their key properties are:

Control	Type	Key properties
rectHeader	Rectangle	X 0, Y 0, Width App.Width, Height 100, teal fill, no border
lblAppName	Label	"Visitor Management System", size 24, bold, white, X 24, Y 20
lblHeaderSubtitle	Label	Size 12, light teal text, Y 59
lblWelcome	Label	Eyebrow text, size 13, bold, Y 130
lblHeroTitle	Label	"Manage visits with ease.", size 30, bold, Y 164, Height 92
lblHeroDescription	Label	Size 14, slate text, Y 260
imgWelcome	Image	X App.Width * 0.59, Y 125, Width App.Width * 0.34, Height 230, ImagePosition.Fit
lblActionHeading	Label	"What would you like to do?", size 19, Y App.Height * 0.47
rectVisitorCard	Rectangle	Y App.Height * 0.56, Height 120, white fill, 1 pixel border
rectRecordsCard	Rectangle	Y rectVisitorCard.Y + 132, same size and style
lblFooter	Label	Y App.Height - 34, size 10, centred

Two design decisions are worth noting. The card contents are positioned relative to their card, for example rectVisitorCard.X + 120, so moving a card moves everything on it. The two buttons also use different styles: the first is solid teal and the second is white with a

teal outline, which tells the user which action is primary.

6. Visit screen redesign

The Visit screen reuses the same theme: mint background, teal header titled "Visit Management", and a "Today's Visits" section heading. The existing galVisits gallery is restyled (position, template size 140, padding 8, no border) with its Items formula left alone, and btnCheckIn gets the same teal styling. The proposed target also shows summary counters (Scheduled, Checked In, Completed) and a search box, but the written steps do not build them, so they remain open items. The chat suggests building this in stages and checking with a screenshot.

Control	Key properties
rectVisitHeader	X 0, Y 0, Width App.Width, Height 100, teal fill
lblVisitTitle	"Visit Management", size 24, bold, white
lblVisitSubtitle	"Manage appointments and visitor check-ins", size 12, Y 60
lblVisitSection	"Today's Visits", size 20, bold, Y 120
galVisits	X 24, Y 170, Width App.Width - 48, Height App.Height - 190, TemplateSize 140, TemplatePadding 8, no border

Inside the gallery, the existing data-bound labels keep their formulas and only change appearance: main text colour, sizes 12 to 14, and a bold size 15 for the visit title. The section heading is a fixed label for now, and the chat says it can be made dynamic later. The assistant proposed doing Steps 1 to 3 first and then asking for a screenshot before styling the gallery fields.

7. Model-Driven App column view

The user asked how to show only the required fields in the table. The answer was to edit the table view in Solutions, keep Visit Name, Visitor, Host, Visit Date, Purpose and Status (optionally Check-In and Check-Out time), then save and publish. This changes only the list view, with no data deleted. Showing fewer fields on an individual record needs a separate change to the Main Form.

The aim is a cleaner list for back-office users, who see only the six columns that matter. The chat points to the active-visits style view as the usual one to edit, and notes that Check-In Time and Check-Out Time can be added if attendance should be visible in the list. Two limits apply: the change affects the list view only, and the individual record form keeps showing every field until its Main Form is edited.

8. Issues encountered

- **Layering of controls:** the rectangle covered the header labels, and the "Send to back" option was not available where first suggested. Fixes offered were the Tree view menu, the toolbar Reorder option, or recreating the rectangle before the labels, since new controls sit on top.
- **Layout dependence:** positions are starting values for a desktop canvas and may need adjusting if the app has a different screen size, for example the footer overlapping the second card.

Habits that prevent both problems:

- Insert background rectangles before the labels that sit on them, because new controls are placed on top.
- Check the order of controls in Tree view after adding each card.
- Do not rename a control that other formulas refer to; change only its look and position.
- Never delete existing controls to fix a layering problem unless the replacement is ready.

9. Observations on the approach

- Changes were made one screen and one step at a time, with the assistant asking for screenshots and control names before giving exact positions.
- Existing logic (formulas, gallery, flow) was protected, which keeps the risk low.
- The chat gives precise property values, which makes the build repeatable, but it relies on the user to apply each one by hand.

The assistant also offered a reminder to run a final visual and navigation test, once after the Home screen guide and again after the Visit screen guide. The user has not yet taken up either offer in the shared chat.

10. Current status

- Home screen: complete
- Visit screen: designed, completion not confirmed

- Model-Driven view: steps given, completion not confirmed
- Records screen: not covered in this chat
- End-to-end testing and email flow: not done yet

Counting these five items, one is confirmed complete, two are in progress and two have not started.

11. Recommended next steps

1. Finish the Visit screen and test that visits appear and Check-In updates the timestamp.
2. Add the summary counters and a search box if they are wanted for the demonstration.
3. Apply the same teal theme to the Records screen so all three screens match.
4. Update the Power Automate flow so the notification email goes to the Host dynamically.
5. Run an end-to-end test: register a visitor, receive the email, check in, and confirm the record in the Model-Driven App.
6. Review the layout on the actual screen size and fix any overlaps before the final demonstration.

Steps 4 and 5 match the follow-ups the assistant offered at the end of the chat: explaining how to test visitor registration and the email notification end to end, guiding an update of the flow so the email goes to the Host dynamically, and checking that the flow is enabled and connected to the Canvas App environment.

12. Testing checklist

These checks come from the test steps in the chat plus the planned end-to-end test.

- [ ] Home: both navigation buttons open the correct screens in Preview
- [ ] Home: header, cards and footer are visible with no overlap
- [ ] Visit: galVisits still shows the existing visits
- [ ] Visit: Check-In on a test record updates the timestamp
- [ ] Visit: status values display correctly on each card
- [ ] Flow: the notification email reaches the right Host
- [ ] Model-Driven: the view shows only the six required columns after publishing
- [ ] App: saved, with no red-underlined formulas

13. Risks and mitigations

Risk	Why it matters	Mitigation
Overlapping controls	Rectangles can cover labels and buttons	Check Tree view order; add backgrounds first
Different screen size	Positions are desktop starting values	Use App.Width and App.Height; test on the real device
Changing existing formulas	Navigation or Check-In could stop working	Change appearance only; keep OnSelect and Items
Renaming referenced controls	Other formulas may break	Rename only controls nothing refers to
Untested flow and email	Notification may fail during the demonstration	Run the end-to-end test before presenting
Record form unchanged	Users still see every field on a record	Edit the Main Form separately if needed

14. Lessons learned

- Build one screen and one step at a time, and ask for a screenshot before positioning controls.
- Separate appearance from behaviour: restyle freely, but treat formulas as protected.
- Define a palette and naming scheme first, so later screens need no new design decisions.
- Position card contents relative to the card so a layout change needs one edit.
- Treat layout values as starting points and confirm them on the actual canvas.

15. Glossary

Term	Meaning
Canvas App	A Power Apps app where the builder places controls freely on screens
Model-Driven App	A Power Apps app generated from Dataverse tables, views and forms

Term	Meaning
Dataverse	The Microsoft Power Platform database that holds the Visit table
Power Automate	The tool that runs the flow behind Check-In and notifications
Gallery	A control that repeats a template for every record in a list
Tree view	The left panel that lists screens and controls and shows their order
OnSelect	The property that holds the formula that runs when a control is clicked
RGBA	A colour formula with red, green, blue and opacity values
HoverFill, PressedFill	Button colours when the pointer is over it or it is clicked
Main Form	The Model-Driven form that shows a single record